Matplotlib

Topic: Data Visualization using Python

Focus: Plots, customization, subplots, figures, saving graphs, and algorithm visualization

1. Introduction

Matplotlib isawidely used Python library for creating static, animated, and interactive visualizations. It is especially useful for turning numerical or categorical data into graphs that are easier to understand.

In engineering and programming projects, visualization is useful for analyzing experimental results, simulation data, mathematical functions, sensor measurements, and algorithms. Matplotlib can also be used to visualize path-planning algorithms such as A*.

2. Why Learn Matplotlib?

LearningMatplotlibhelpsdevelop theability to communicate numerical information visually. A graph can reveal trends, relationships, peaks, changes, and unusual values that may be difficult to notice in raw numbers.

For an engineering student, Matplotlib is particularly useful for plotting signals, comparing experiments, displaying simulation results, analyzing data, and creating visual demonstrations of algorithms.

3. Installing Matplotlib

IfMatplotlib isnotalreadyinstalled,itcan usually be installed with pip:

pip install matplotlib

In many Python environments, Matplotlib is already available. It is commonly imported through the pyplot interface.

4. Importing Matplotlib

Themostcommonimport is:

import matplotlib.pyplot as plt

The name **plt** is a conventional short alias. The pyplot module provides functions for creating figures, plotting data, adding labels, and displaying or saving graphs.

5. Basic Line Plot

Alineplotconnectsdatapoints and is useful for showing trends or relationships between two numerical variables.

6. Basic Plot Example

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x,  y,  marker="o")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Basic Line Plot")
plt.grid(True)
plt.show()
```

7. Important Plot Components

Aprofessionalgraph normally containsseveralcomponents: a title, axis labels, appropriate scales, and—when multiple datasets are present—a legend. Grid lines may also improve readability. Common functions include **plt.plot()**, **plt.xlabel()**, **plt.ylabel()**, **plt.title()**, **plt.legend()**, **plt.grid()**, and **plt.show()**.

8. Scatter Plot

Ascatterplot displaysindividual observations as points. It is useful for examining relationships between two variables and identifying patterns or clusters.

9. Bar Chart

Abarchart represents categorical or discrete values using rectangular bars. It is useful for comparing quantities across categories.

10. Histogram

Ahistogram groupsnumerical values into intervals called bins. It is useful for understanding the distribution of a dataset.

11. Pie Chart

Apiechart displays proportions of a whole. It can be useful for a small number of categories, although bar charts are often easier to compare precisely.

12. Customizing a Graph

Matplotlibprovides manywaysto customize a graph. You can change line properties, markers, labels, limits, legends, grid settings, and figure dimensions.

Common properties include linewidth, linestyle, marker, marker size, transparency, axis limits, and figure size.

13. Multiple Lines on One Graph

Multipledatasets canbeplotted onthesameaxes.This is useful for comparing functions, measurements, or experimental results.

14. Subplots

Subplotsallow multiple graphs to be arranged inside a single figure. They are useful when several related results need to be compared together.

15. Figure and Axes

A**Figure** is theoverallcanvas orwindow. An **Axes** object represents an individual plotting area inside the figure. Understanding this object-oriented structure becomes useful for larger projects and complex

visualizations.

16. Object-Oriented Matplotlib

For simple plots, pyplot functions are convenient. For larger programs, an object-oriented approach using **fig, ax = plt.subplots()** often makes the code easier to organize and maintain.

17. Plotting Mathematical Functions

Matplotlib can be combined with numerical Python tools to visualize mathematical functions. A common workflow is to generate x-values, calculate corresponding y-values, and plot y against x.

18. Saving Figures

A graph can be saved to an image file instead of only displaying it on the screen. The **plt.savefig()** function can save figures in formats such as PNG, JPEG, SVG, and PDF, depending on the environment and requirements.

19. Example: Save a Figure

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("x squared")
plt.title("Quadratic Relationship")

plt.savefig("quadratic_plot.png", dpi=300, bbox_inches="tight")
plt.show()
```

20. Visualization of A* Pathfinding

Matplotlib is especially useful for visualizing A* because a grid can represent an environment. Different visual elements can represent free cells, obstacles, start and goal positions, explored cells, and the final path.

This kind of visualization makes it easier to understand how an algorithm searches the environment and can also help identify implementation errors.

21. Example: A* Grid Visualization

```
import matplotlib.pyplot as plt

grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)
path = [(0,0), (1,0), (2,0), (2,1),
        (2,2), (3,2), (3,3), (4,3), (4,4)]
```

```
plt.imshow(grid,  origin="upper")
py = [r for r, c in path]
px = [c for r, c in path]

plt.plot(px, py, marker="o")
plt.scatter(start[1], start[0], label="Start")
plt.scatter(goal[1], goal[0], label="Goal")

plt.title("A* Path Visualization")
plt.legend()
plt.show()
```

22. Animation

Matplotlibcan also beused to create animations. For algorithm demonstrations, animation can show the search process one step at a time. This can make the behavior of an algorithm much easier to understand.

23. Common Matplotlib Functions

Function / Concept	Purpose
plt.plot()	Create a line plot
plt.scatter()	Create a scatter plot
plt.bar()	Create a bar chart
plt.hist()	Create a histogram
plt.pie()	Create a pie chart
plt.xlabel()	Label the x-axis
plt.ylabel()	Label the y-axis
plt.title()	Add a title
plt.legend()	Display a legend
plt.grid()	Display grid lines
plt.xlim() / plt.ylim()	Set axis limits
plt.figure()	Create/select a figure
plt.subplots()	Create figure and Axes objects
plt.savefig()	Save a figure
plt.show()	Display the figure